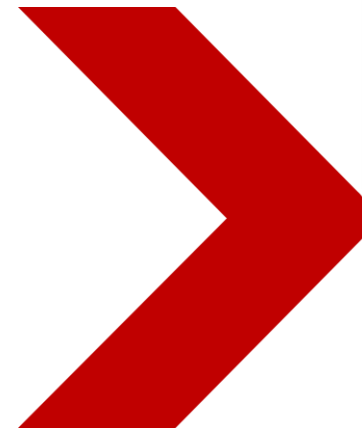


Java程序设计基础

Java Programming Fundamentals

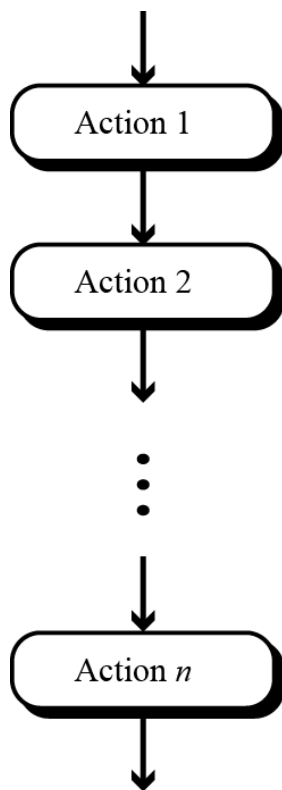
Control Flow Statements



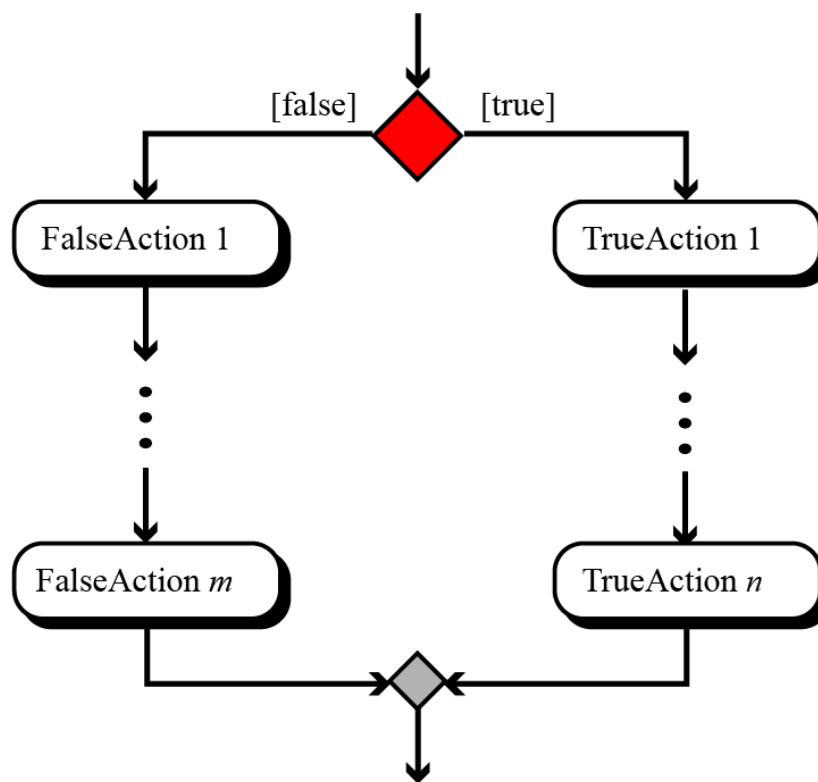
Topics

- 程序的基本结构
- 分支结构
- 循环结构

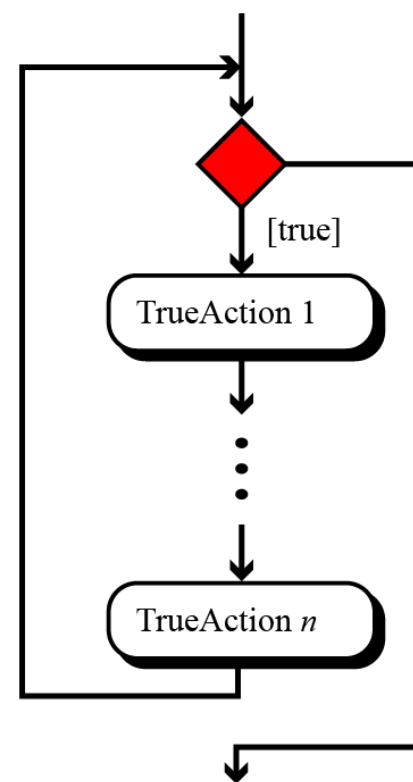
程序的基本结构



a. Sequence



b. Decision



c. Repetition

程序结构的伪代码描述

```
action 1  
action 2  
...  
action  $n$ 
```

a. Sequence

```
if (condition)  
{  
    trueAction(s)  
}  
else  
{  
    falseAction(s)  
}
```

b. Decision

```
while (condition)  
{  
    Action(s)  
}
```

c. Repetition

判断数字的奇偶性

- 输入1个整数，判断该数是奇数还是偶数。
- 分析

读入一个整数

if (该数能被2整除)

则该数为偶数

else

该数为奇数

number % 2 == 0

> 二分支结构和 if-else 语句

if (表达式)

语句1

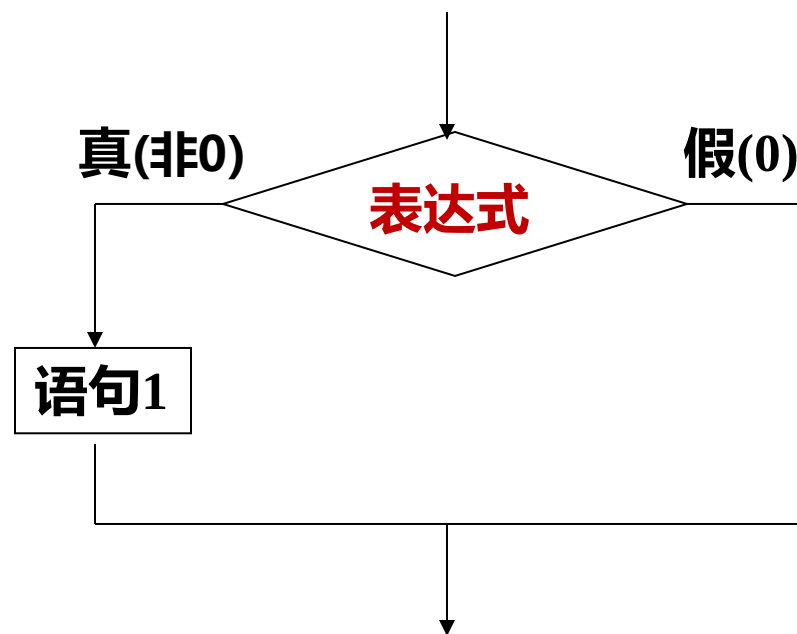
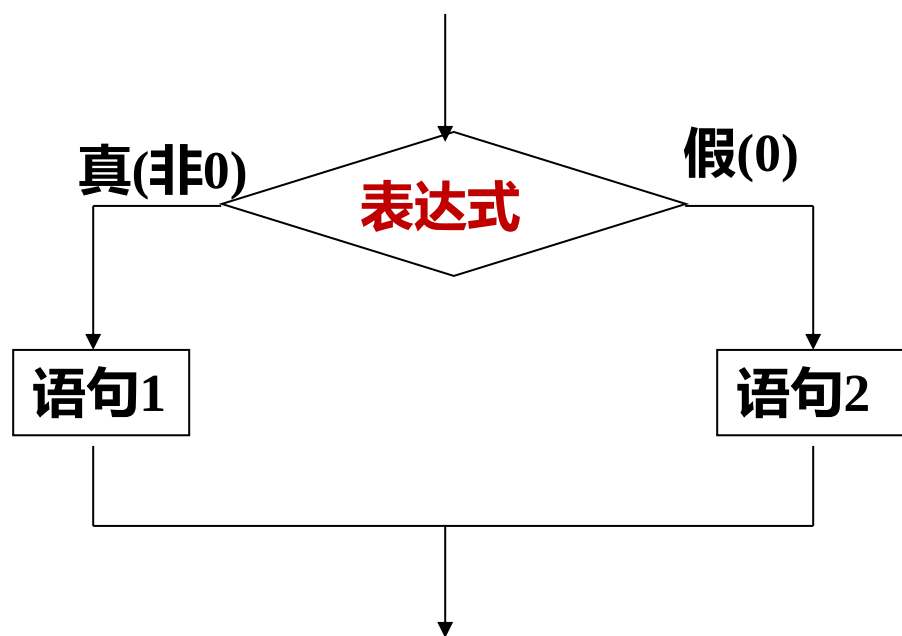
else

语句2

变形
只有if分支

if (表达式)

语句1



> 嵌套的 if – else 语句

if (表达式1)

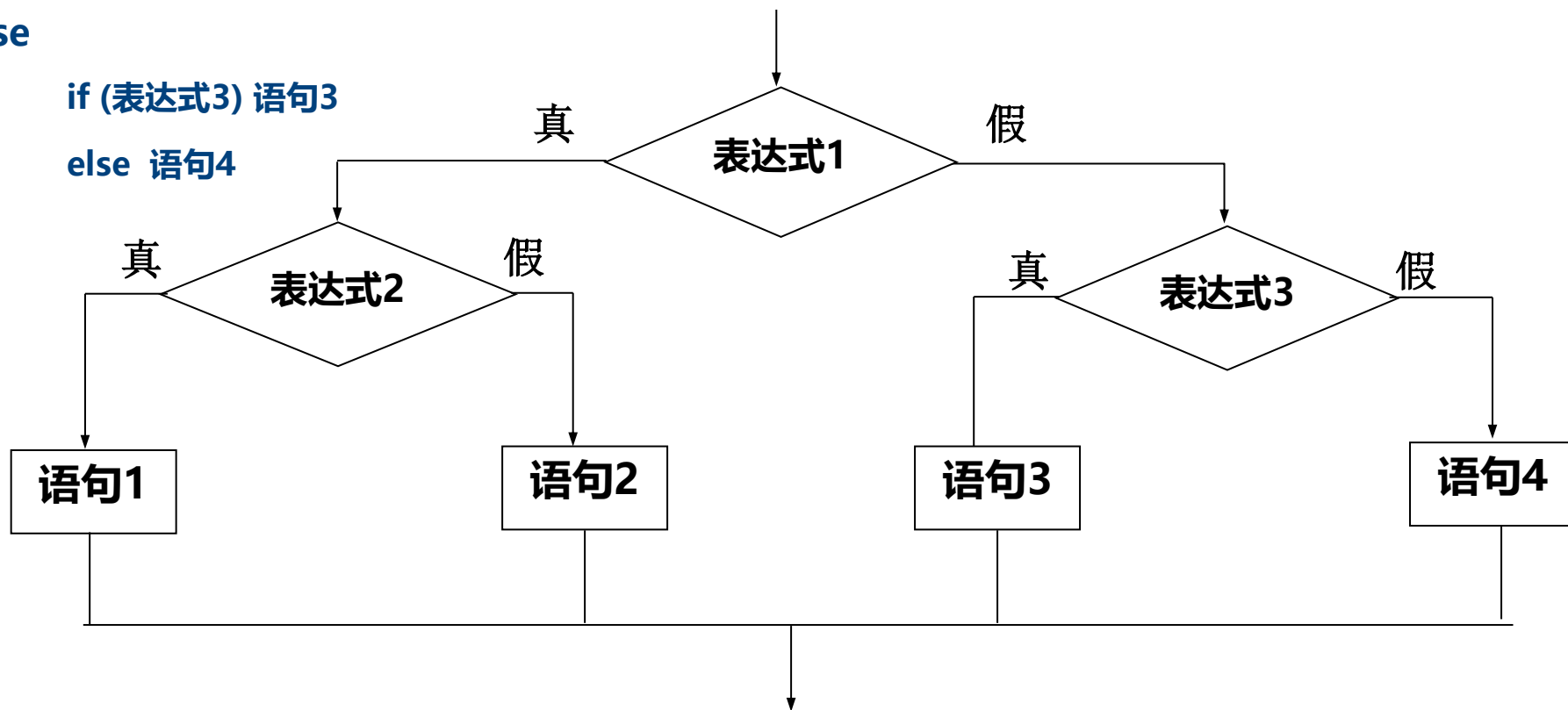
 if (表达式2) 语句1

 else 语句2

else

 if (表达式3) 语句3

 else 语句4



多分支结构和else – if 语句

else-if 语句是最常用的实现多分支（多路选择）的方法

if (表达式1)

语句1;

else if (表达式2)

语句2;

.....

else if (表达式n-1)

语句n-1;

else

语句n;

switch-case

```
switch(表达式){  
    case 常量表达式1: 语句段1; break;  
    case 常量表达式2: 语句段2; break;  
    .....  
    case 常量表达式n: 语句段n ; break;  
    default :          语句段n+1; break;  
}
```

选择（分支）结构实现方法总结

- 分支结构一般分为二分支和多分支两种结构
- 二分支结构用基本的 if 语句实现
- 多分支结构的实现方法：
 - else – if 语句
 - switch语句
 - 嵌套的 if - else语句

> for语句

for (表达式1; 表达式2; 表达式3)

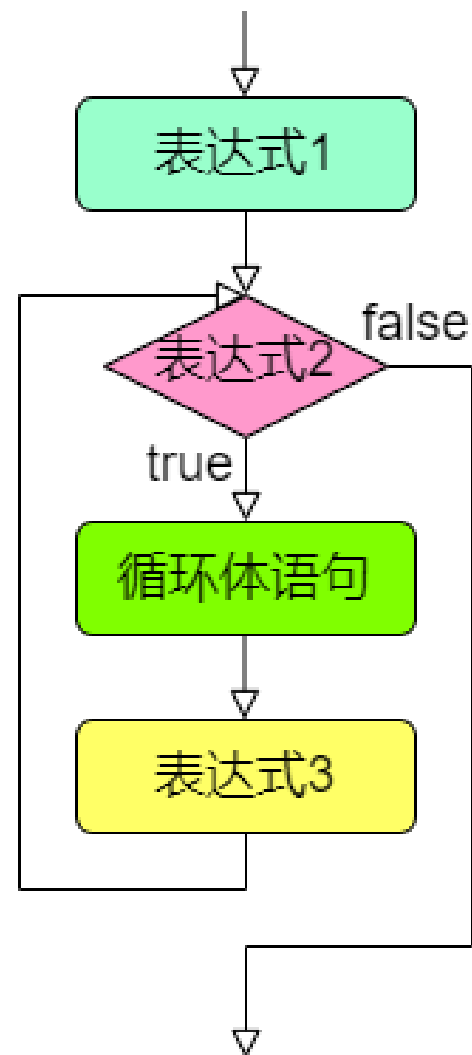
循环体语句;

实现语句的重复执行

3个表达式、循环体语句

☑ 书写顺序和执行顺序不同

☑ 表达式1只执行一次



> for语句中的循环变量

for (表达式1; 表达式2; 表达式3)
循环体语句;

- **for**通过改变和判断循环(控制)变量的值来控制循环的执行

赋初值	判断其值	改变其值
for (int i = 1 ;	i <= 100 ;	i = i + 1)
{		
Sum = Sum + i;		循环体语句
}		

for语句的说明

```
for ( int i = 1 ; i<=100 ; i=i+1 )  
{  
    Sum = Sum + i;  
}
```

表达式1：给循环变量赋初值，指定循环的起点。

$i = 1$

表达式2：给出循环的条件，判断循环是否达到终点？

$i \leq 100$

表达式3：设置循环的步长，改变循环变量的值，从而可改变表达式2的真假性。

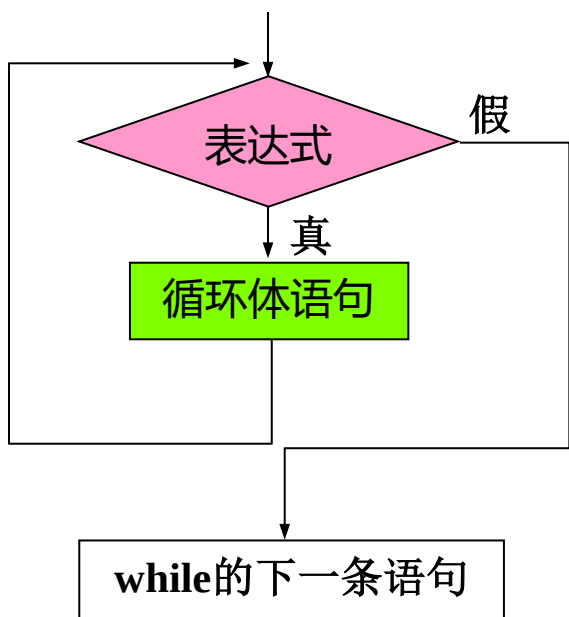
$i = i + 1$

循环体语句：被反复执行的语句，一条语句。

$Sum = Sum + i$

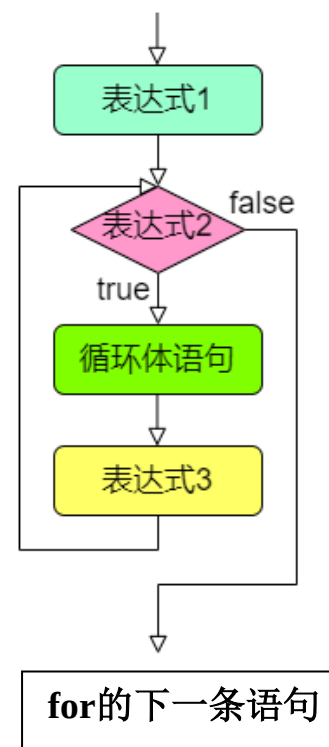
> while 语句

while (条件)
循环体语句;



循环条件

循环体



while与for都是在循环前先判断条件

> for → while

```
for (表达式1; 表达式2; 表达式3)
    循环体语句
```



```
表达式1;
while (表达式2) {
    for的循环体语句;
    表达式3;
}
```

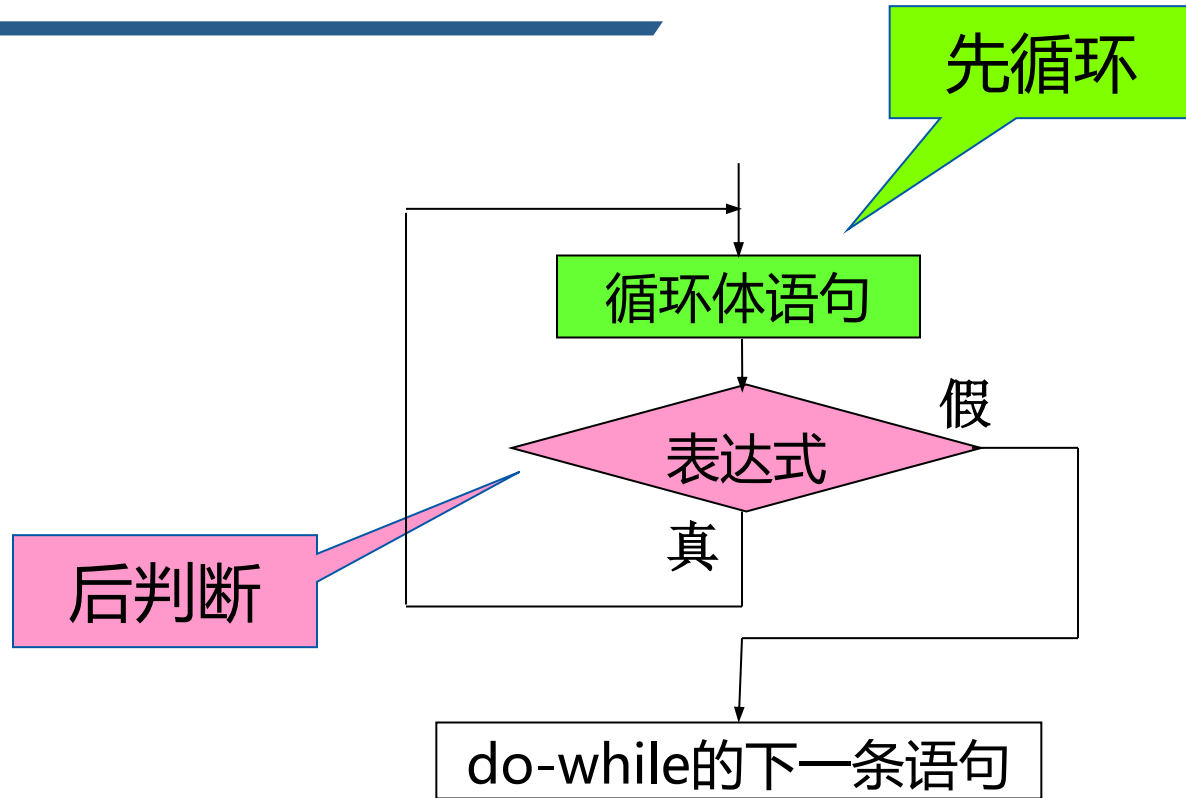
```
for (int i = 1; i <= 10; i++){
    sum = sum + i;
}
```



```
int i = 1;           /* 循环变量赋初值 */
while (i <= 10){     /* 循环条件 */
    sum = sum + i;
    i++;             /* 循环变量的改变 */
}
```

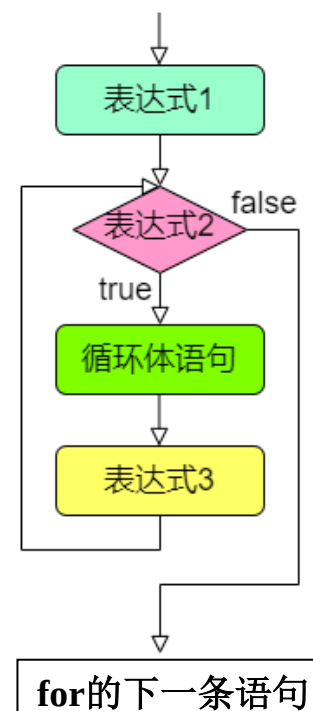
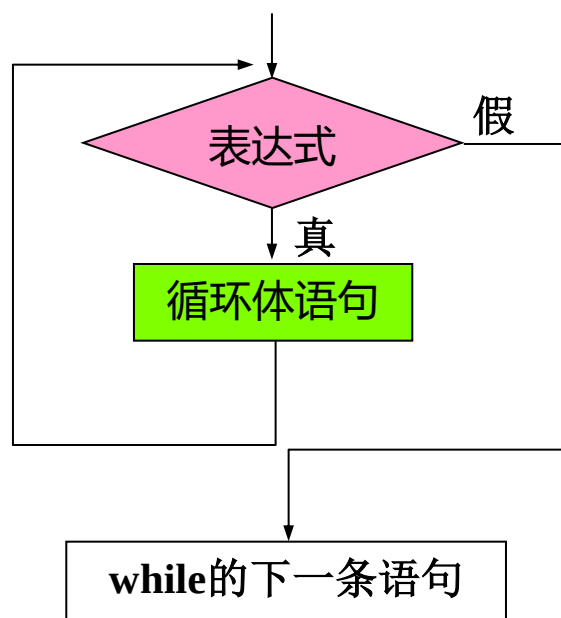
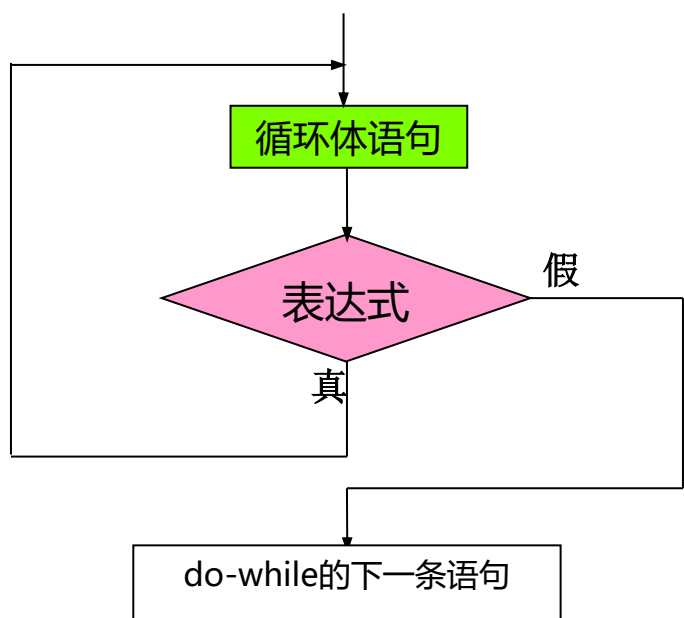
> do - while 语句

```
do{  
    循环体语句;  
} while (表达式)
```



> while 和 do-while、for 的比较

- **while**: 先判别条件, 再决定是否循环;
- **do-while**: 先至少循环一次, 然后再根据条件决定是否继续循环。



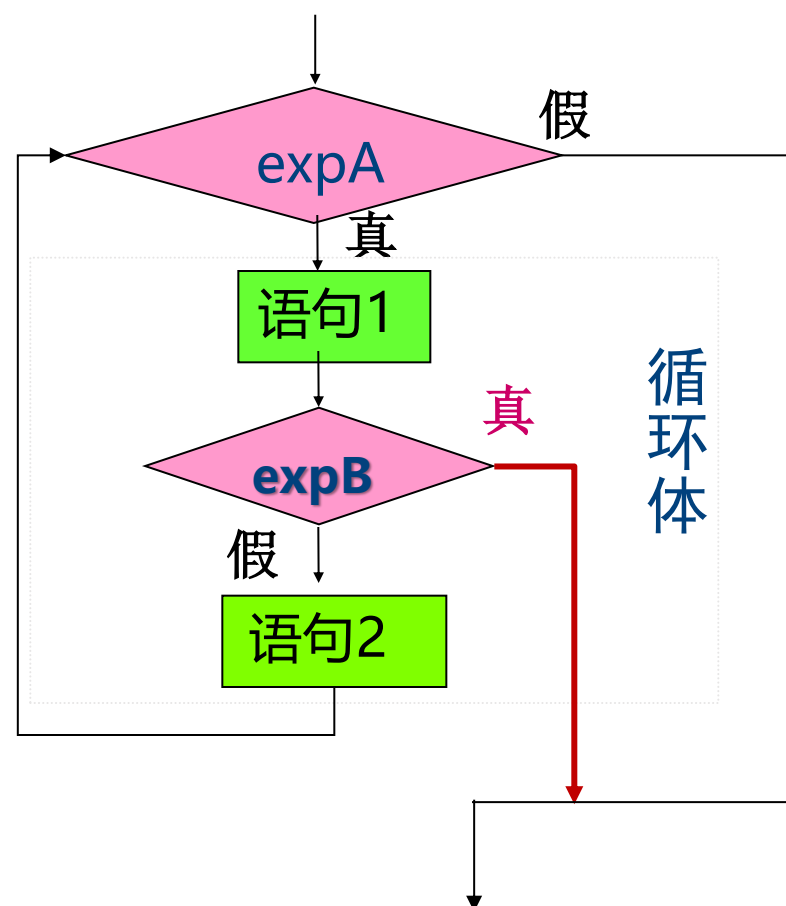
> break 语句

```
while ( expA ){  
    语句1  
    if ( expB )  
        break;  
    语句2  
}
```

当循环有多个出口时:

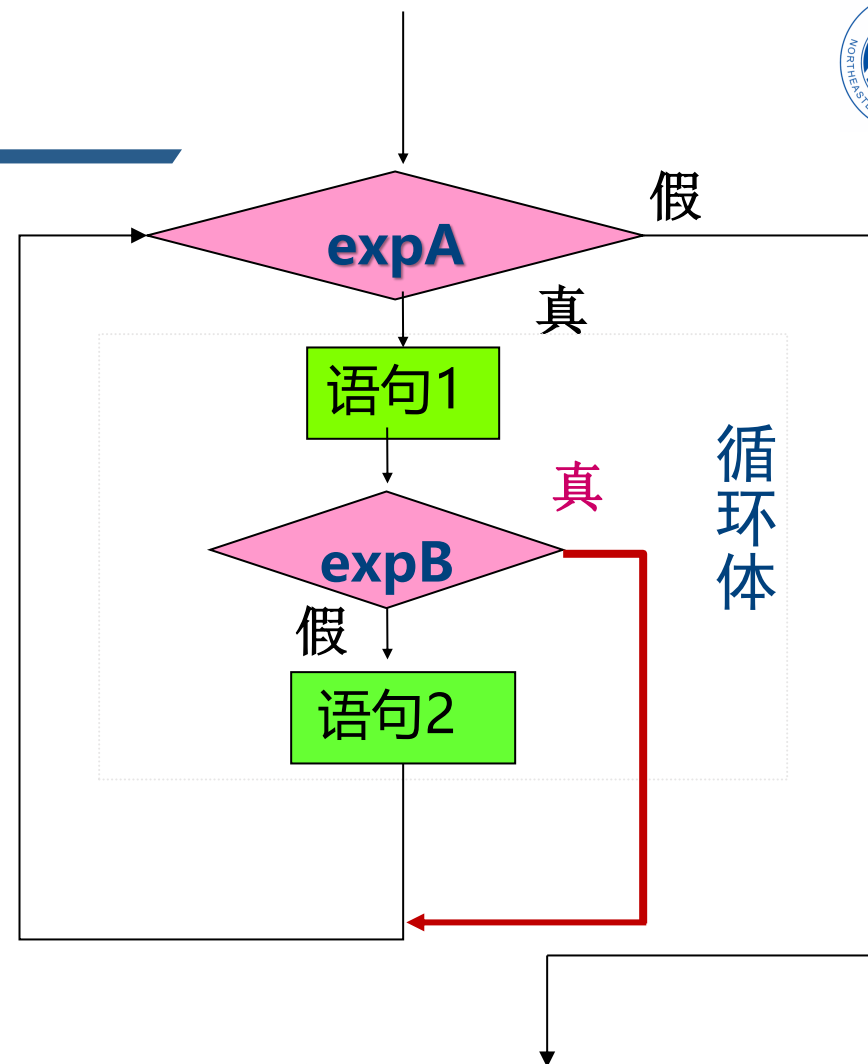
- 共同表示循环条件
- 区分结束条件

```
for ( i = 2; i <= m/2; i++)  
    if ( m % i == 0 ) break;  
if ( i > m/2 ) printf("Yes!");  
else printf("No!\n");
```



> continue 语句

```
while (expA){  
    语句1  
    if (expB)  
        continue;  
    语句2  
}
```



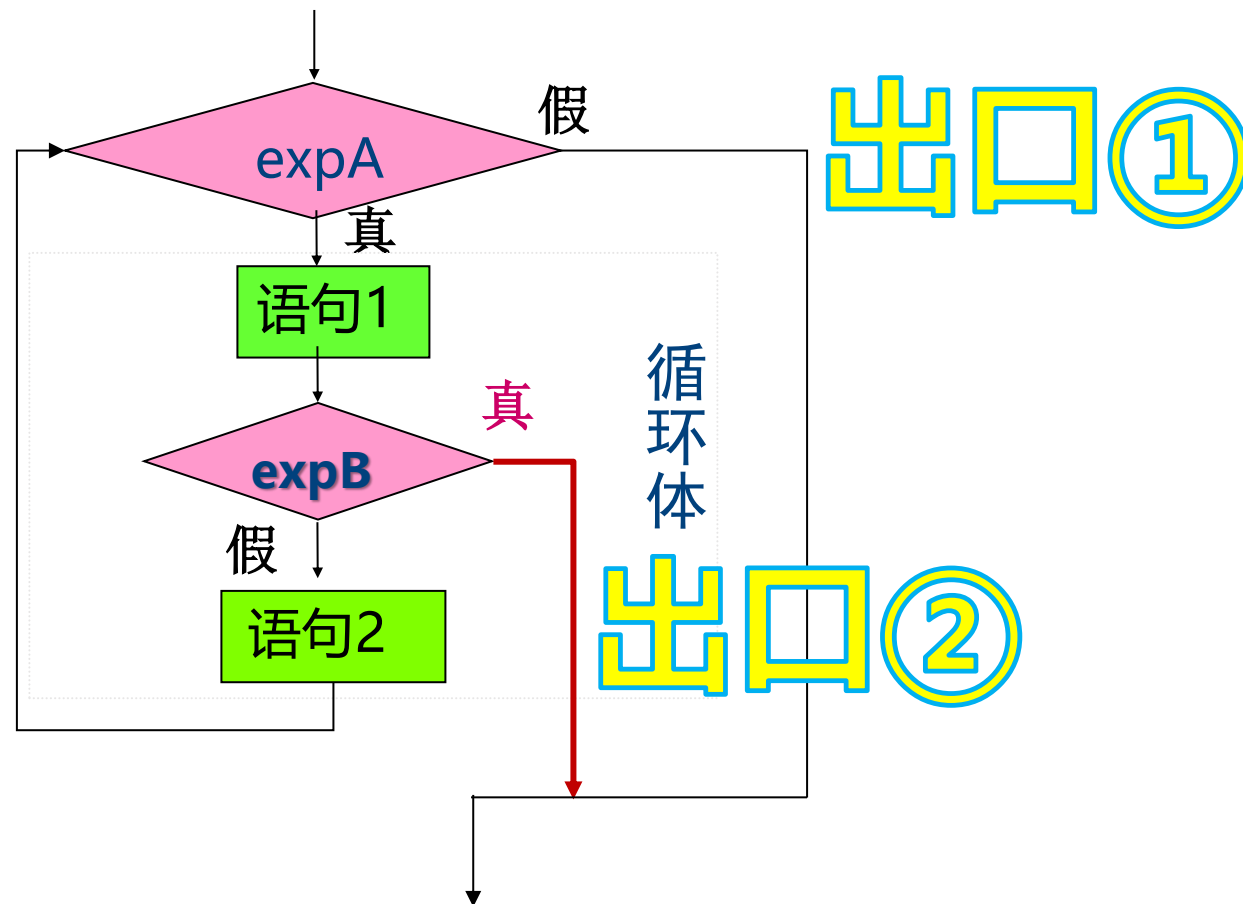
跳过continue后面的语句，继续下一次循环

break 语句带来的多出口判断问题

```
while ( expA ){  
    语句1  
    if ( expB )  
        break;  
    语句2  
}
```

当循环有多个出口时：

- 考虑引入一些标志变量flag
- 便于后续处理时区分出口



带标签的break

```
public class Break01 {  
    public static void main(String[] args) {  
        // 外侧循环  
        for (int i = 1; i <=5 ; i++) {  
            System.out.println("外侧循环:"+i);  
            // 内侧循环  
            for (int j = 1; j <=5 ; j++) {  
                if(j == 3){  
                    break;// 作用于内侧循环  
                }  
                System.out.println("---内侧循环:"+j);  
            }  
        }  
    }  
}
```

```
1 public class Break02 {  
2     public static void main(String[] args) {  
3         // 外侧循环  
4         outer: for (int i = 1; i <=5 ; i++) {  
5             System.out.println("外侧循环:"+i);  
6             // 内侧循环  
7             for (int j = 1; j <=5 ; j++) {  
8                 if(j == 3){  
9                     break outer;// 作用于外侧循环  
10                }  
11                System.out.println("---内侧循环:"+j);  
12            }  
13        }  
14    }  
15 }  
16 }
```



带标签的continue

```
1 public class Continue01 {
2     public static void main(String[] args) {
3         // 外侧循环
4         for (int i = 1; i <=5 ; i++) {
5             System.out.println("外侧循环:"+i);
6             // 内侧循环
7             for (int j = 1; j <=5 ; j++) {
8                 if(j == 3){
9                     continue ;// 作用于内侧循环
10                }
11                System.out.println("---内侧循环:"+j);
12            }
13        }
14    }
15 }
16 }
```

```
1 public class Continue02 {
2     public static void main(String[] args) {
3         // 外侧循环
4         outer: for (int i = 1; i <=5 ; i++) {
5             System.out.println("外侧循环:"+i);
6             // 内侧循环
7             for (int j = 1; j <=5 ; j++) {
8                 if(j == 3){
9                     // 需求: 在此处, 我们需要使用continue来作用于外侧循环, 如何实现???
10                    continue outer;// 作用于外侧循环, 这里等效于break
11                }
12                System.out.println("---内侧循环:"+j);
13            }
14        }
15    }
16 }
17 }
```

循环结构程序设计

- 循环程序的实现要点
 - 归纳出哪些操作需要反复执行？ 循环体
 - 这些操作在什么情况下重复执行？ 循环条件
- 常见的循环控制方式
 - 计数控制、计算值控制、输入值控制
 - 多重控制（计数控制+计算值控制，.....）
- 选用合适的循环语句
for / while / do-while

常见的循环控制方式

- 计数控制:
sum = 0;
for (i = 1; i <= n; i++){
sum = sum + fact (i);
}
- 计算值控制:
item = 1.0;
while (fabs (item) >= 0.0001) {
item = flag * 1.0 / denominator;
.....
}
- 输入值控制为主:
scanf ("%lf", &grade);
while (grade >= 0) {
.....
scanf ("%lf", &grade);
}
- 计数控制+计算值控制:

循环语句的选择

- 主要考虑因素-循环控制方式
 - 事先给定循环次数, 首选 for
 - 通过其他条件控制循环, 考虑 while或 do-while

if (循环次数已知)

使用 **for** 语句

else /* 循环次数未知 */

if (循环条件在进入循环时明确)

使用 **while** 语句

else /* 循环条件需要在循环体中明确 */

使用 **do-while** 语句